

DFTL: A Flash Translation Layer Employing Demand-based Selective Caching of Page-level Address Mappings

발표자 서혜원

발표날짜 2026.08.06

Design of DFTL

→ 앞서 full merge방식때문에 하이브리드 방식은 성능이 저하된다는 점을 확인

DFTL이 제안한 방식:

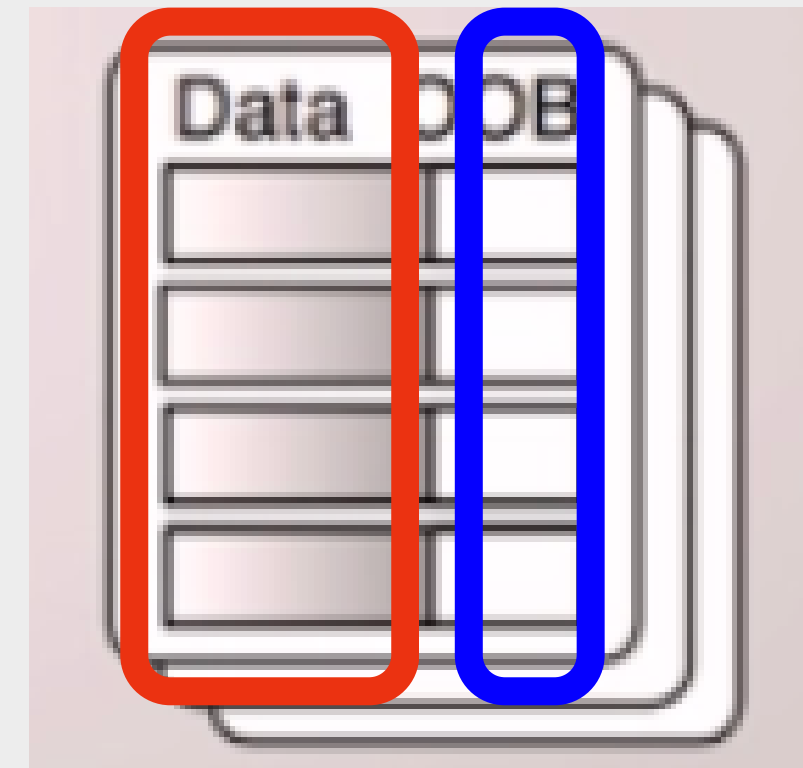
All paged-mapping + temporal locality이용한 caching

→ 이 방식은 제한된 SRAM 크기를 잘 사용할 특별한 주소 변환 방식이 필요

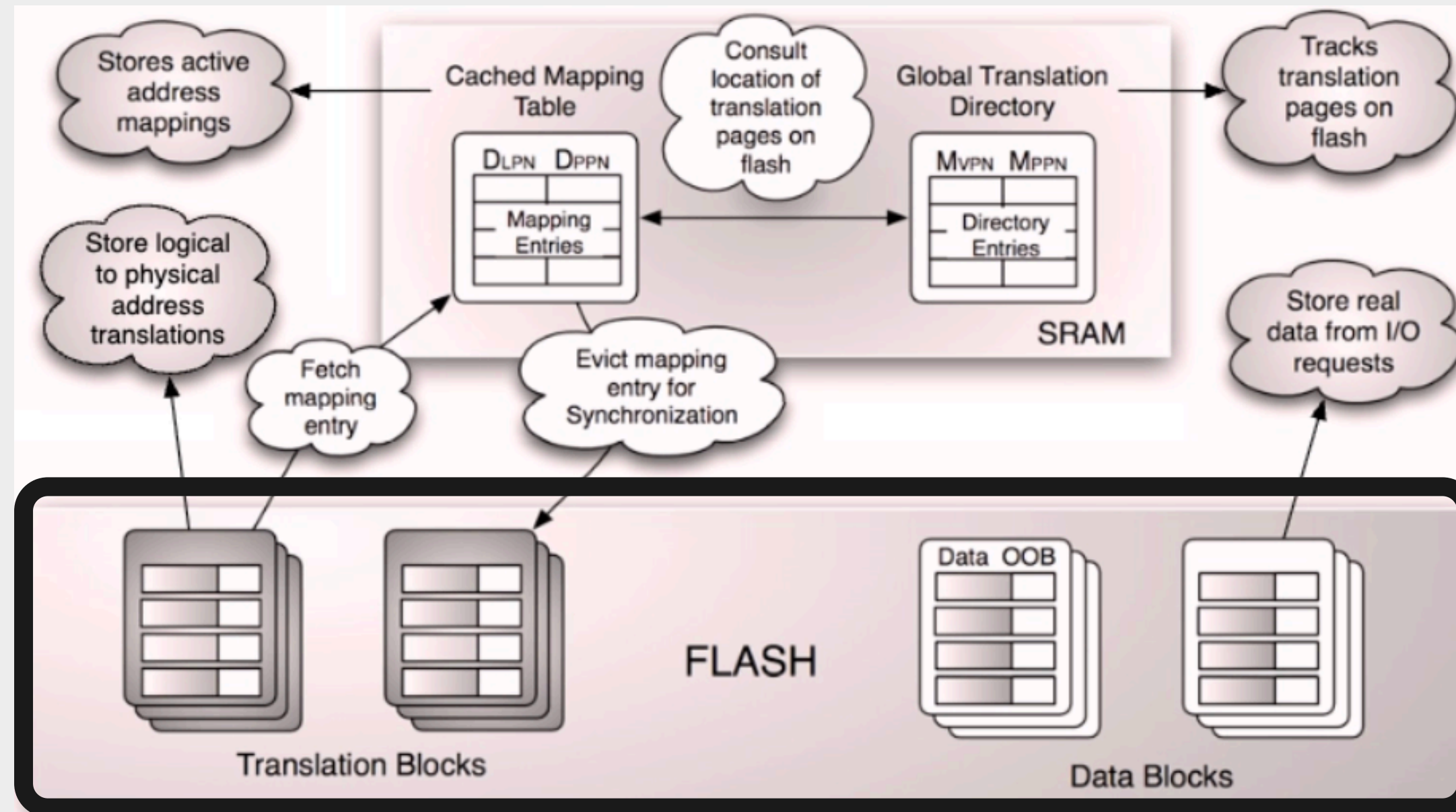
DFTL Architecture

-mapping table의 전체 이미지를 어디에 저장?

- **Data Area**에 저장
 - OOB area보다 Data area의 공간이 더 큼
 - 1번의 write/read 작업으로 훨씬 많은 양의 mapping 정보를 가져올 수 있음
 - 모든 mapping 정보를 저장하는데 필요한 크기는 2MB (1GB의 0.2%)
- **OOB(Out-Of-Band)** 즉 spare area에 저장
 - data를 가져오기 위해 여러번의 접근 필요

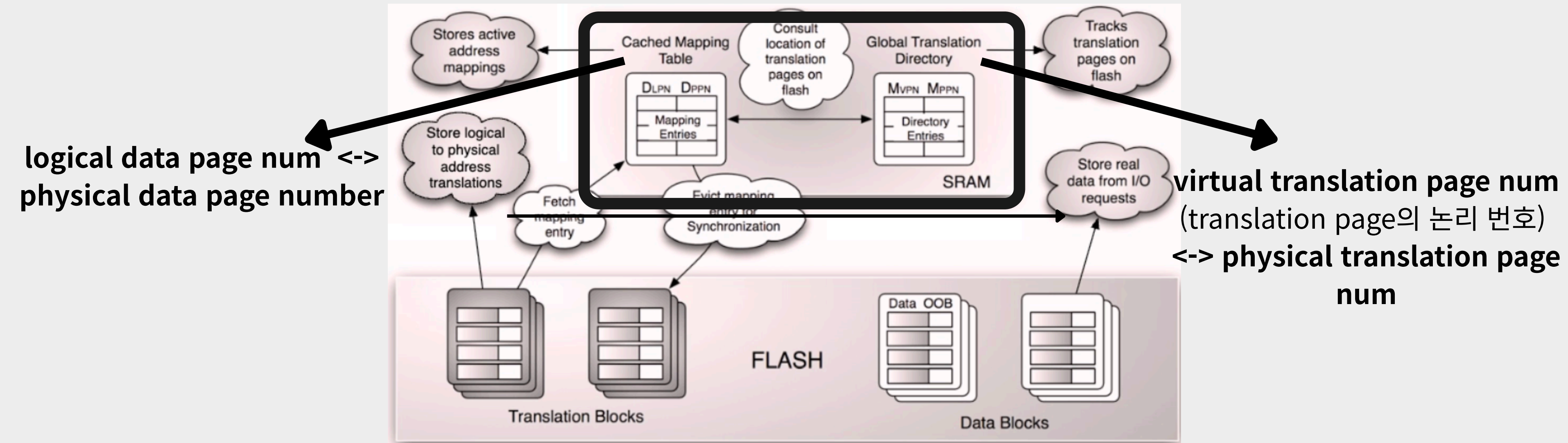


DFTL Architecture - Flash 영역



- **Data pages** : 읽기/쓰기 작업 중에 접근되거나 업데이트되는 실제 데이터 저장
- **Translation pages** : logical<->physical address에 대한 정보만 저장

DFTL Architecture - SRAM 영역



- **CMT(Cached Mapping Table)** : sram에 존재하는 활성 매핑들
- **GTD(Global Translation Directory)** : out-of-place update로 인해 흩어진 translation pages를 추적하는 매핑테이블

GTD(Global Translation Directory)

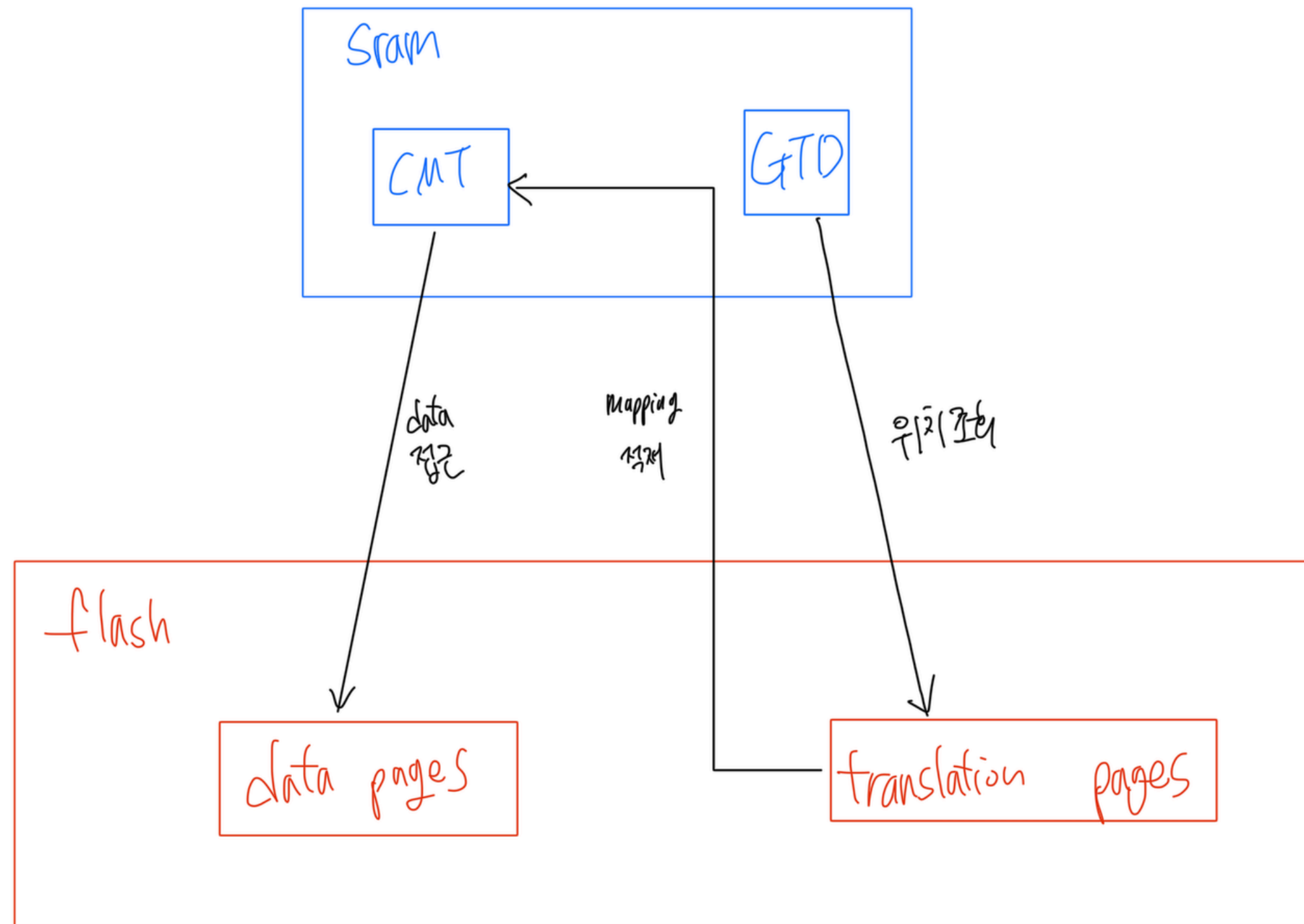
<out-of-place update 방식>

1. erased된 페이지에 새 data 씀
2. 원본data 페이지 invalid로 표시
3. 매핑정보 업데이트

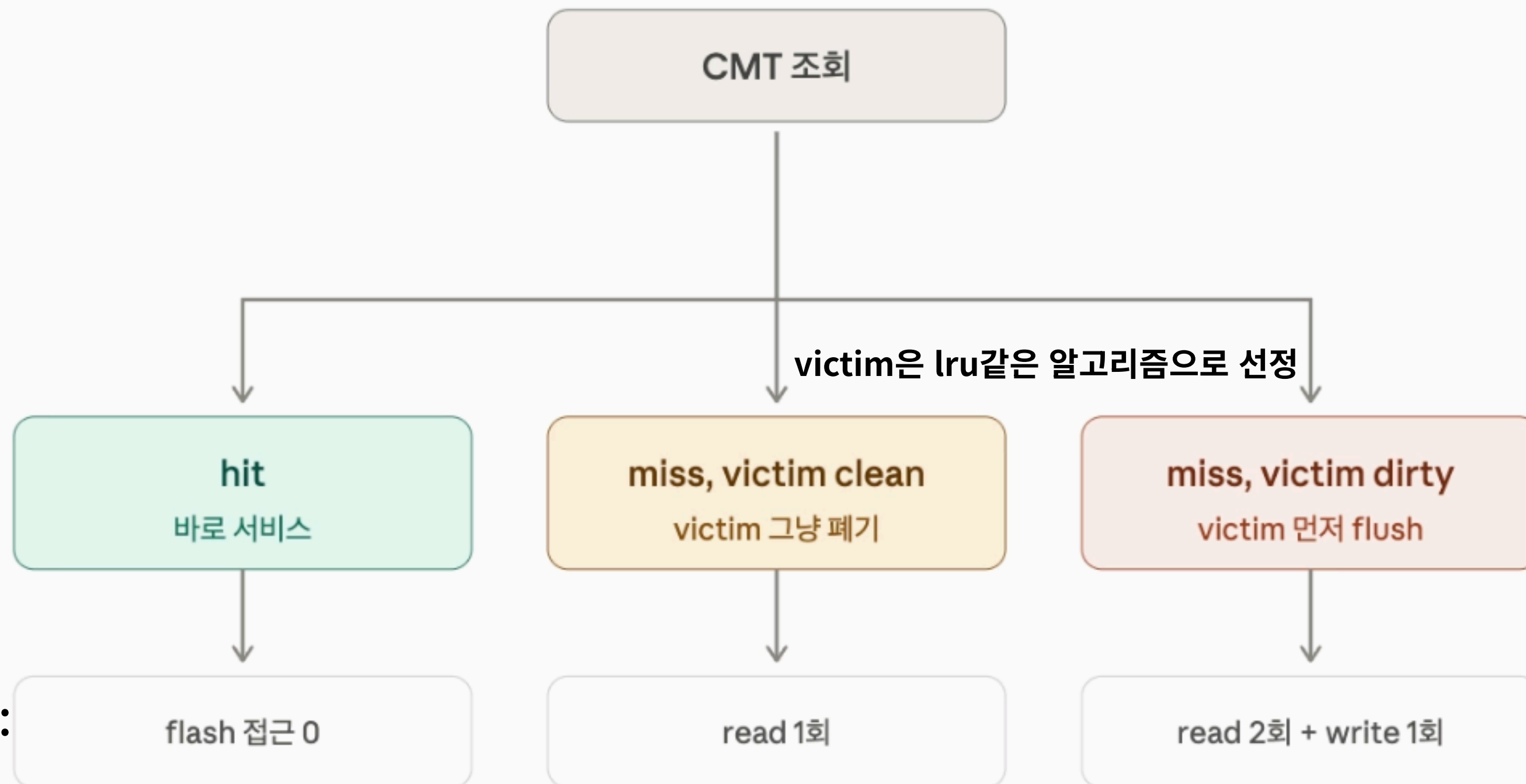
- 이 과정에서 매핑정보를 담은 페이지들도 매번 새로운 physical address로 옮겨짐
- translation pages는 전체 플래시 메모리에 물리적으로 흩어지게 됨
- GTD를 이용하여 모든 변환페이지 추적

+1GB의 플래시의 경우 4KB의 GTD가 필요 → 큰 공간 오버헤드를 유발하지 X

DFTL Architecture



주소변환 process



요청1개당:

flash 접근 0

read 1회

read 2회 + write 1회

주소변환 process - miss/victim clean

clean의 의미: sram에 올라온 이후 매핑 엔트리가 바뀐 적이 없는 victim
(플래시 메모리 정보 == sram 정보)

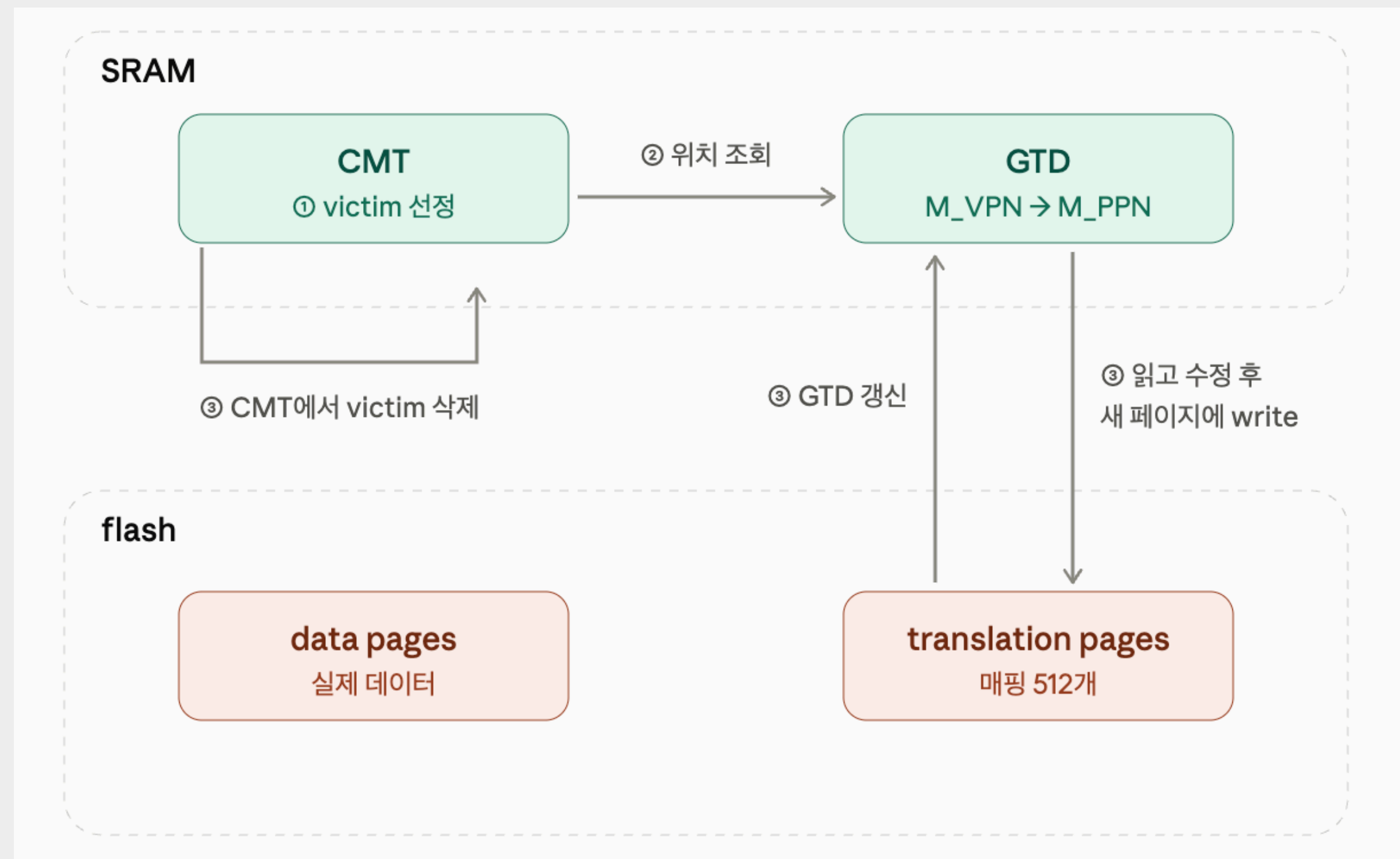
=> clean 상태의 victim은 flash memory에 이미 최신정보가 저장되어 있으므로
바로 eviction 시키면 됨

주소변환 process - miss/victim dirty

dirty의 의미: sram에 올라온 이후 매핑 엔트리가 바뀐 적이 있는 victim
(플래시 메모리 정보 != sram 정보), sram정보가 최신

<victim 처리>

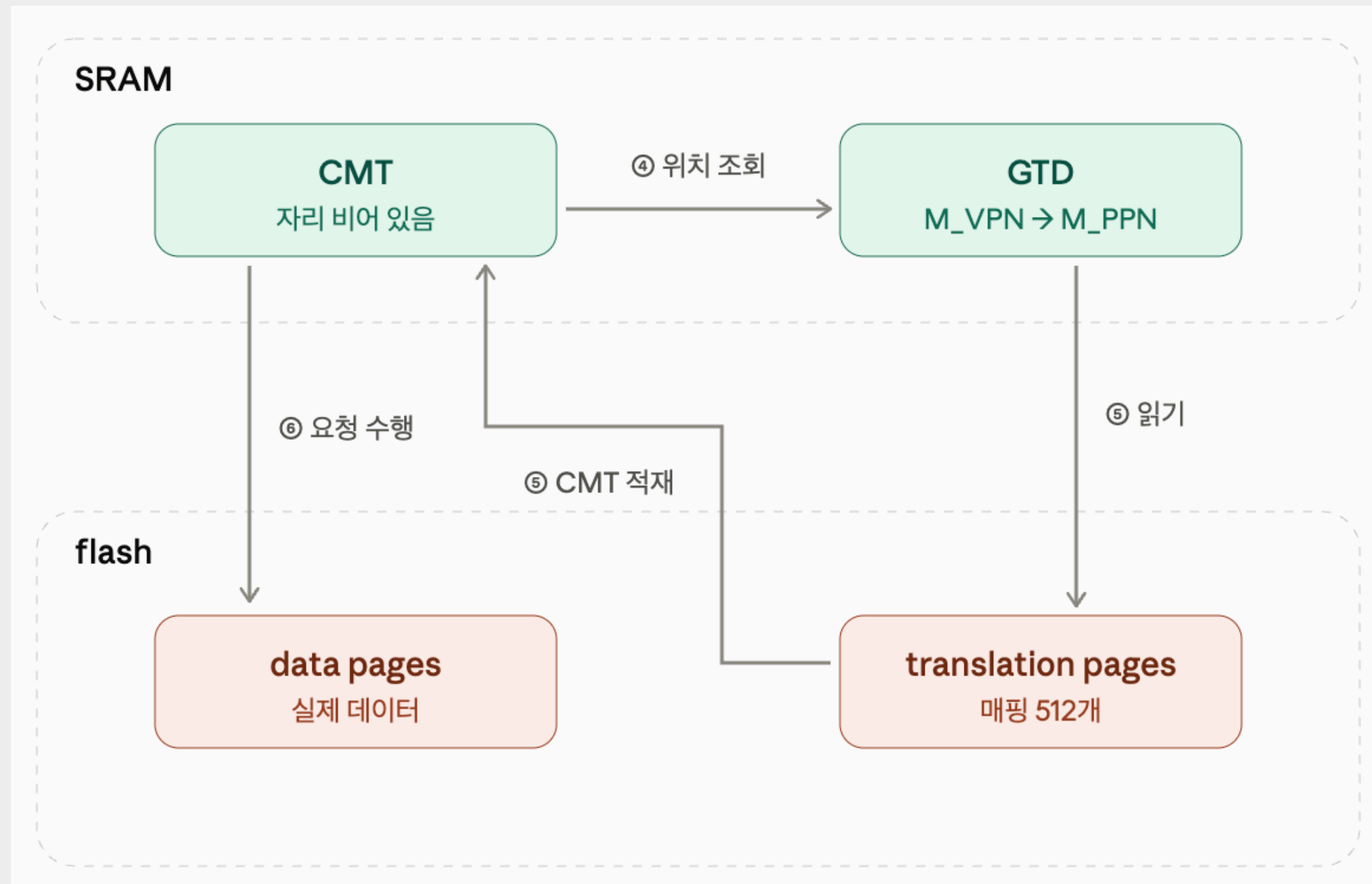
victim의 단위: CMT entry 1개



주소변환 process - miss/victim dirty

dirty의 의미: sram에 올라온 이후 매핑 엔트리가 바뀐 적이 있는 victim
(플래시 메모리 정보 != sram 정보), sram정보가 최신

<새 요청 처리>



Garbage Collection

시간이 지날수록 erased 상태인 block이 지워짐

→ DFTL은 GCthreshold라는 high watermark를 유지하여, 이를 초과하면 GC를 호출

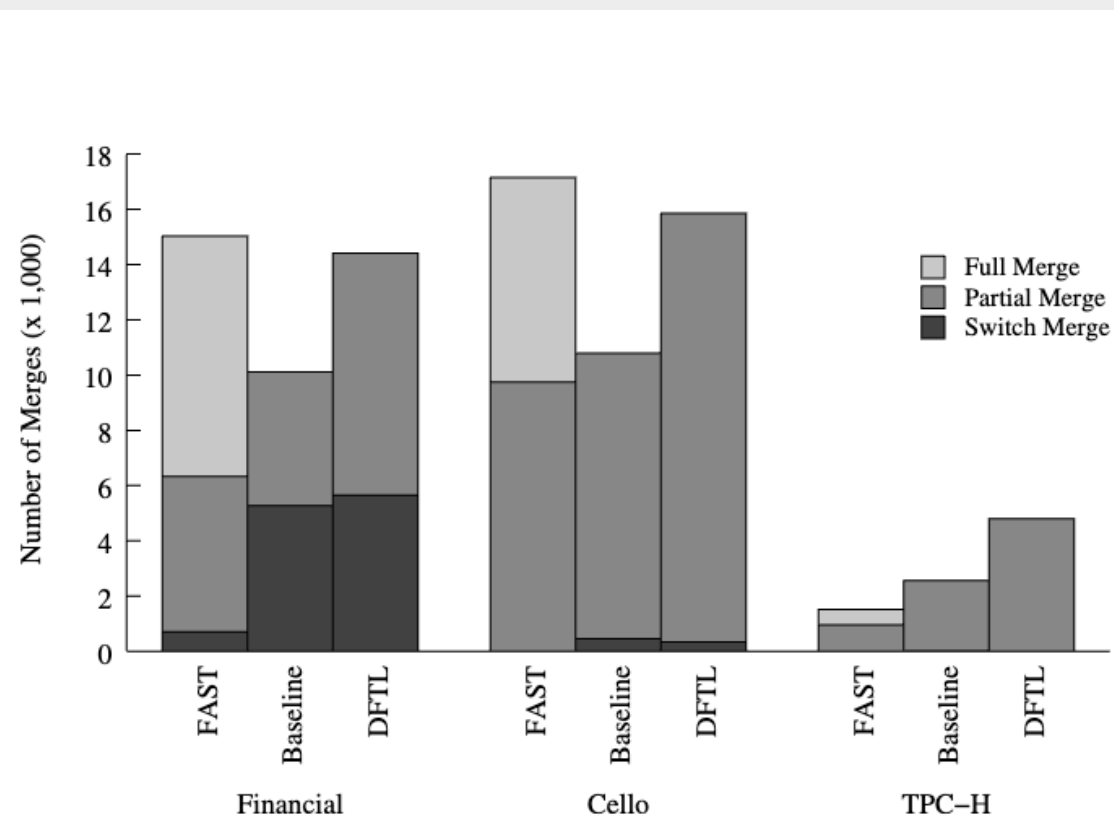
victim block의
종류

data block: 실제 데이터를 담은 공간이 victim이 되면 단순히 data를 옮기는 것을 넘어 매핑정보까지 바뀜!

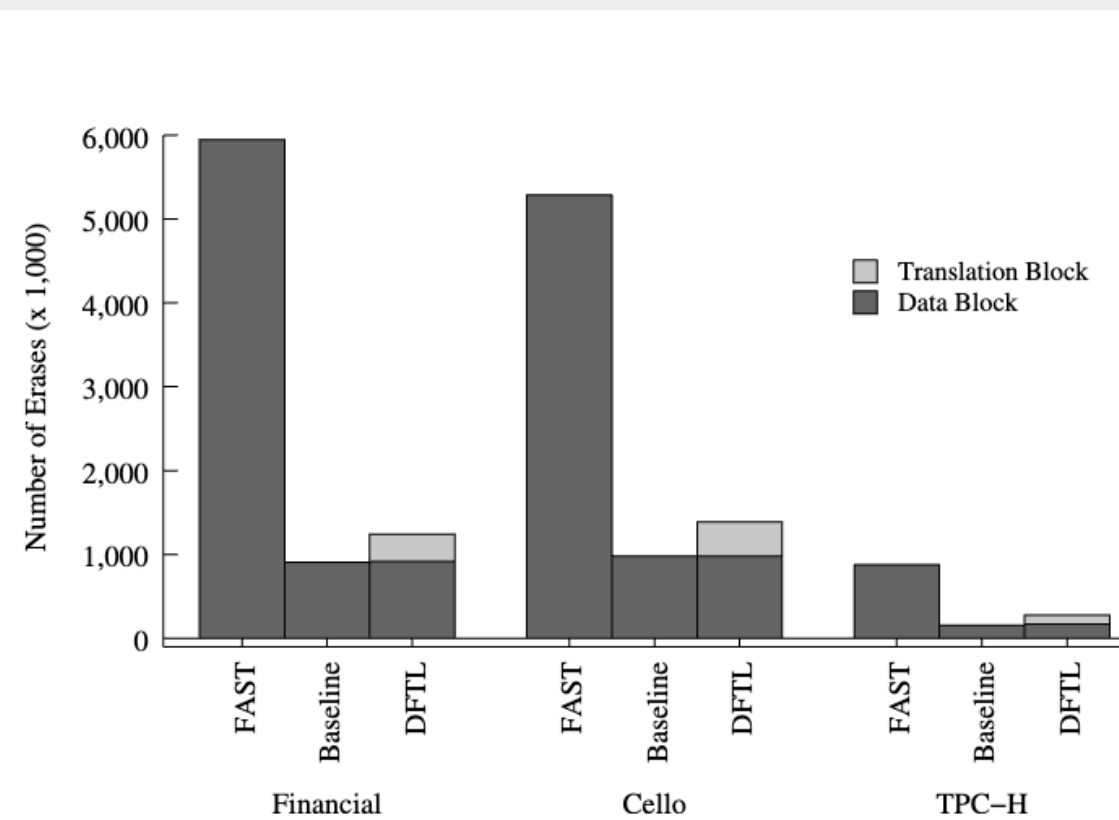
**valid page를 current data block으로 복사
& 옮긴 page와 관련된 모든 translation page 및 CMT entry update**

translation block: 매핑정보만 바꾸면 됨

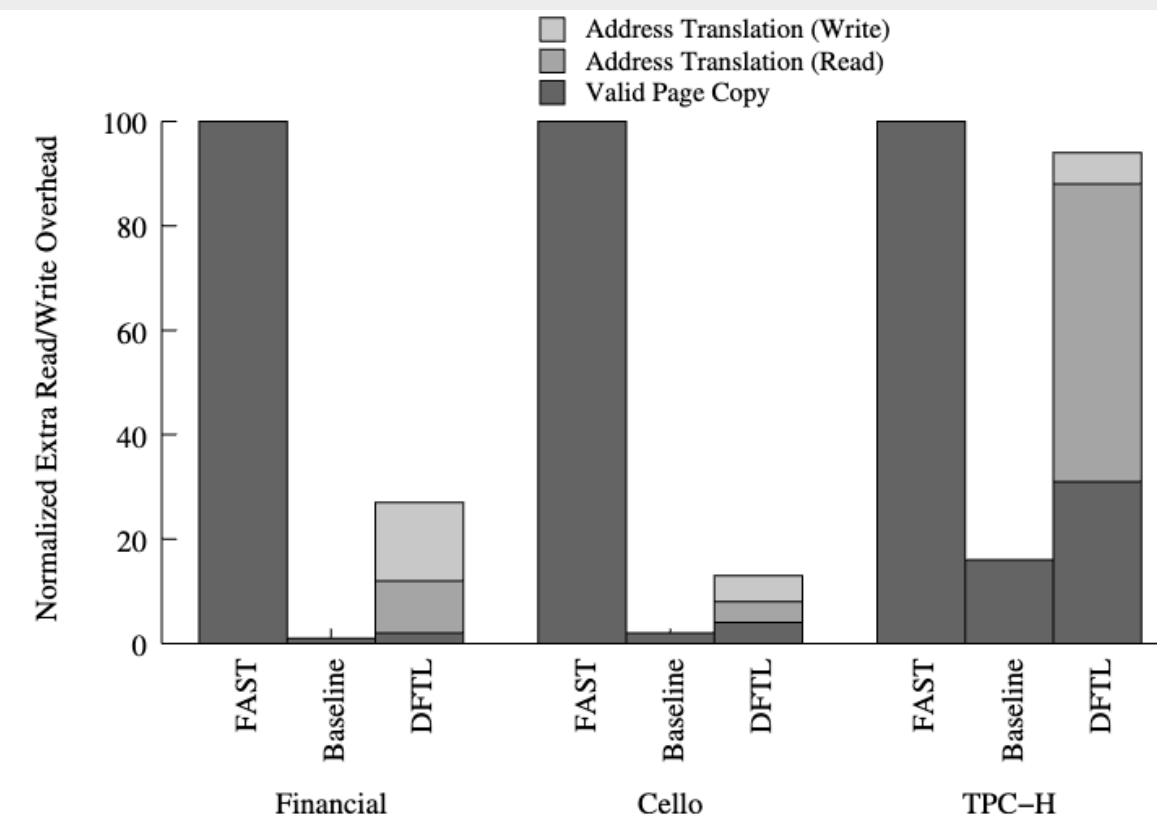
**valid page를 current translation block으로 복사
& GTD 업데이트**



(a) Merge Operations



(b) Block Erases



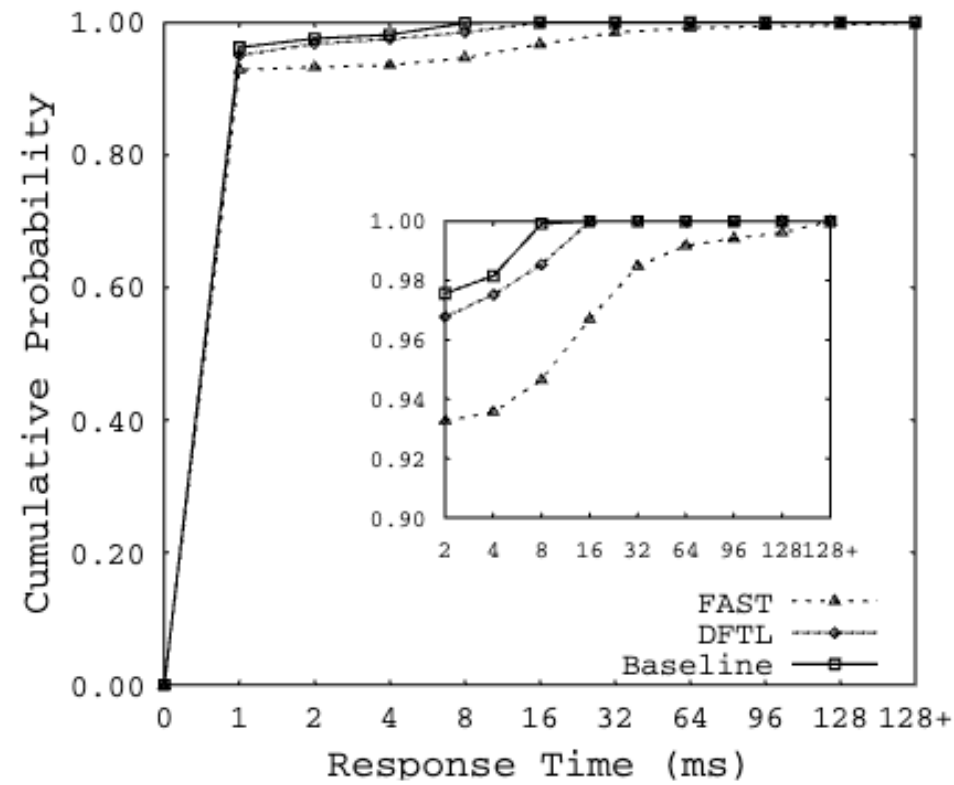
(c) Extra Read/Write Operations

DFTL 성능 실험결과

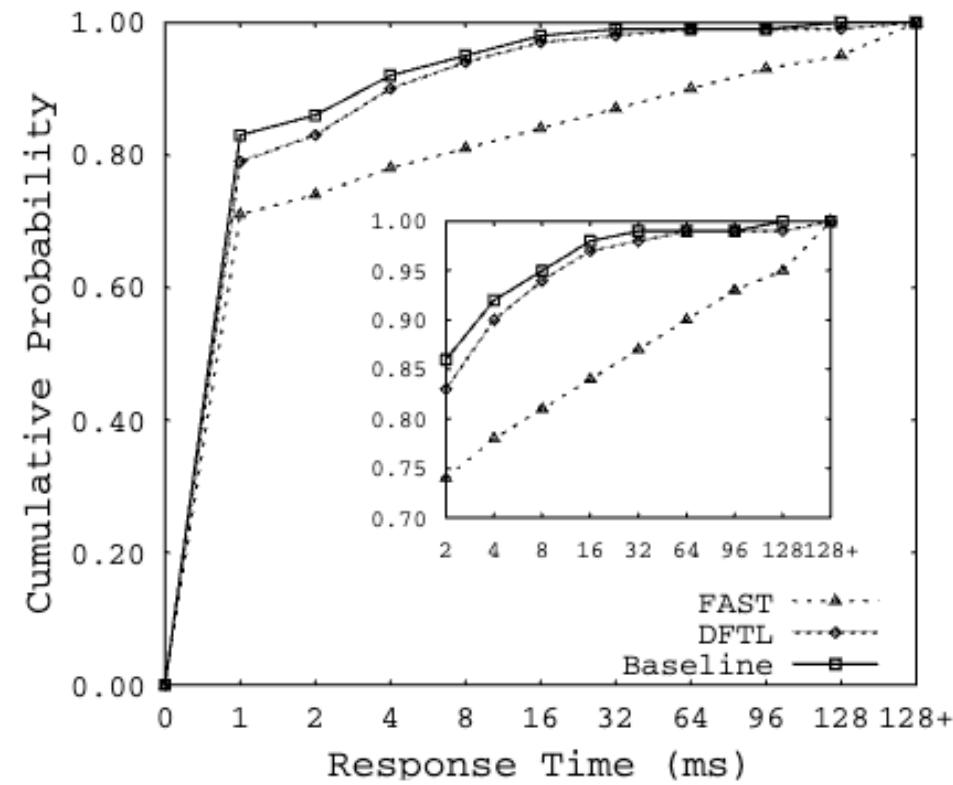
- 비교대상: **FAST(hybrid FTL) vs DFTL vs Baseline(sram이 무한&all page mapping)**
- workload 4개: **Financial·Cello99 (write 많음) / TPC-H·Web Search (read 많음)**
- 조건: SRAM은 FAST가 겨우 돌아갈 최소치로 고정, hybrid에는 log buffer 3% 추가 제공

x축: 응답시간

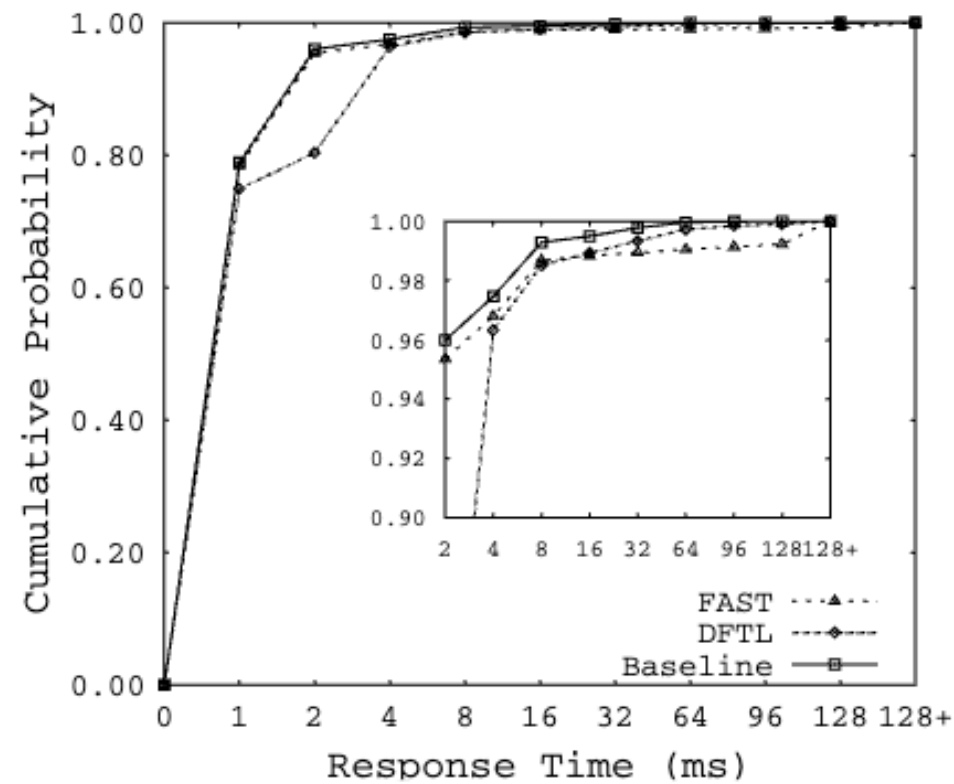
y축: 응답시간이 xms이하인 요청의 비율



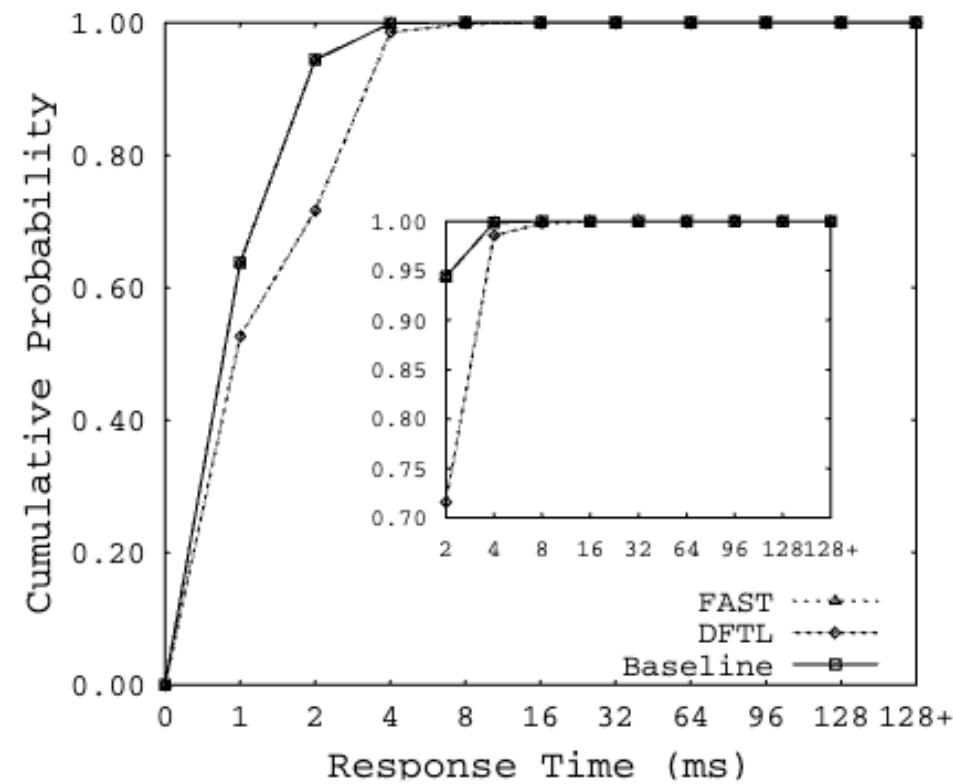
(a) Financial Trace (OLTP)



(b) Cello99



(c) TPC-H



(d) Web-Search

A: FAST는 느림요청이 많다는 뜻

C: 1~2ms 구간을 보면 FAST가 DFTL위에 있지만 오른쪽으로 갈 수록 1.0에 도달하지 못함 → DFTL이 56% 더 나옴

THANK YOU